



Department of Computer Science and Engineering
2024 CSE BATCH
23CSE463 – Quantum Computing

NAME : Agneay B Nair
REG NO : CH.SC.U4CSE24102

WORKSHEET



WORKSHEET- 1

EX.NO : 1

DATE :

TITLE : Implementation and Verification of Single-Qubit, Two-Qubit and Three-Qubit Quantum Gates and Circuits Using Qiskit

AIM:

To construct and simulate quantum circuits using single-qubit, two-qubit and three-qubit gates in Qiskit, to verify each gate against its matrix and truth table, and to prepare the entangled Bell and GHZ states.

APPARATUS REQUIRED:

- Python 3.10 or above
- Qiskit 2.x and Qiskit Aer simulator (pip install qiskit qiskit-aer)
- Matplotlib and pylatexenc (for circuit diagrams and histograms)
- Jupyter Notebook / Visual Studio Code (VS Code)
- Internet Connection (for package installation and documentation access)
- Quantum Computing Reference Material or Lab Manual

THEORY:

A qubit is the basic unit of quantum information. Its state is written as

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, |\alpha|^2 + |\beta|^2 = 1$$

where α and β are complex numbers called amplitudes. A measurement gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. A **quantum gate** changes the state of one or more qubits. A gate acting on n qubits is a $2^n \times 2^n$ unitary matrix U , which means $U^\dagger U = I$. This keeps the total probability equal to 1 and makes every gate reversible.

Single-qubit gates. The Pauli-X gate swaps $|0\rangle$ and $|1\rangle$, so it is the quantum NOT gate. The Pauli-Z gate leaves $|0\rangle$ unchanged and changes the sign of $|1\rangle$. The Pauli-Y gate does both, with a factor of i . The S and T gates add phases of $\pi/2$ and $\pi/4$ to $|1\rangle$. The Hadamard gate creates an equal superposition:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, H|0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, H|1\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$



Two-qubit gates. A controlled gate acts on a target qubit only when the control qubit is $|1\rangle$. The CNOT (controlled-X) gate maps $|a, b\rangle$ to $|a, a \oplus b\rangle$, where a is the control, b is the target and $\oplus$ is addition modulo 2 (XOR). With the basis order $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ and the control written first, its matrix is

$$\text{CNOT} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

The CZ gate multiplies $|11\rangle$ by -1 and leaves the other three basis states unchanged. The SWAP gate exchanges the states of two qubits. A Hadamard gate on one qubit followed by a CNOT produces a Bell state, in which the two qubits are **entangled**: neither qubit has a state of its own, and their measurement results always agree.

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$

Three-qubit gates. The Toffoli gate (CCX, controlled-controlled-NOT) flips the target only when both controls are 1: $|a, b, c\rangle \rightarrow |a, b, c \oplus ab\rangle$. With the target starting at 0 it computes the AND of the two controls, so any classical logic circuit can be built reversibly from Toffoli gates. The Fredkin gate (CSWAP, controlled-SWAP) swaps two target qubits only when the control is 1. Each is an 8×8 matrix that rearranges the basis states. A Hadamard gate followed by two CNOT gates gives the three-qubit GHZ state:

$$|\text{GHZ}\rangle = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

Quantum hardware offers only one-qubit and two-qubit gates, so a three-qubit gate is decomposed before it is run. The standard decomposition of the Toffoli gate uses six CNOT gates.

Bit ordering in Qiskit. Qiskit numbers qubits from the right: qubit 0 is the rightmost bit of a printed result. A two-qubit circuit with X on qubit 0 prints 01. Most textbooks write the first qubit on the left, so the matrix Qiskit prints for `cx(0, 1)` has its rows in a different order from the matrix above, although it is the same gate. State which ordering is used whenever a hand calculation is compared with Qiskit output.

PROCEDURE:

1. Install Qiskit and the Aer simulator with `pip install qiskit qiskit-aer matplotlib pylatexenc`, and print the installed version with `qiskit.__version__`.
2. Open Jupyter Notebook or Visual Studio Code.
3. Import `QuantumCircuit` and `transpile` from `qiskit`, `Statevector` and



Operator from `qiskit.quantum_info`, AerSimulator from `qiskit_aer` and `plot_histogram` from `qiskit.visualization`.

4. Create a `QuantumCircuit` with the required number of qubits (one, two or three) and, where a measurement is needed, the same number of classical bits.
5. Prepare the required input state with X gates, then apply the gate under study (x, y, z, h, s, t, cx, cz, swap, ccx, cswap).
6. Draw the circuit with `qc.draw("mpl")` before running it.
7. Display the exact state with `Statevector(qc)` and the gate matrix with `Operator(qc)`. Do this before any measurement is added.
8. Add measurements and run the circuit on AerSimulator with 1024 shots: `sim.run(transpile(qc, sim), shots=1024).result().get_counts()`. Plot the counts with `plot_histogram`.
9. Repeat for every input basis state and fill in the truth table of the gate.
10. Compare the result with the hand calculation. Remember that Qiskit prints qubit 0 as the rightmost bit. Record the code, the circuit diagram, the output and a one-line inference under each program.

PROGRAM:

Common setup (imports and helper functions used by every program):

```
# Common setup for all programs in this worksheet
%matplotlib inline
import qiskit, numpy as np, matplotlib.pyplot as plt
from qiskit import QuantumCircuit, transpile
from qiskit.quantum_info import Statevector, Operator
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
from IPython.display import display

print("Qiskit version:", qiskit.__version__)
sim = AerSimulator()
_seed = 2024 # seeds incremented per run ->
reproducible counts
np.set_printoptions(precision=3, suppress=True, linewidth=110)
plt.rcParams["figure.dpi"] = 100

def show(fig):
    """display a matplotlib figure (circuit diagram / histogram) and close it"""
    display(fig); plt.close(fig)

def run(qc, shots=1024):
    global _seed
    _seed += 1
    return sim.run(transpile(qc, sim), shots=shots,
seed_simulator=_seed).result().get_counts()

def amps(sv, tol=1e-9):
    """print the non-zero amplitudes of a statevector with Qiskit labels (qubit 0
rightmost)"""
    n = sv.num_qubits
```



```
for i, a in enumerate(sv.data):
    if abs(a) > tol:
        print(f" |{i:0{n}b}> : {a.real:+.3f}{a.imag:+.3f}j    P =
{abs(a)**2:.3f}")
```

Qiskit version: 2.5.2

Program 1: Apply the X, Y, Z, H, S and T gates, one at a time, to $|0\rangle$ and to $|1\rangle$. Display the resulting statevector and the gate matrix for each, and compare them with the matrices in the theory.

Code:

```
# Program 1: single-qubit gates X, Y, Z, H, S, T on  $|0\rangle$  and  $|1\rangle$ 
gates = ["x", "y", "z", "h", "s", "t"]
for g in gates:
    U = QuantumCircuit(1)
    getattr(U, g)(0)
    print(f"=== {g.upper()} gate ===")
    print("Matrix (Operator):\n", np.round(Operator(U).data, 3))
    for inp in "01":
        qc = QuantumCircuit(1)
        if inp == "1":
            qc.x(0) # prepare  $|1\rangle$ 
        qc.compose(U, inplace=True)
        print(f" {g.upper()}|{inp}> =", np.round(Statevector(qc).data, 3))
    print()

# circuit diagram: all six gates acting on  $|1\rangle$ , separated by barriers
demo = QuantumCircuit(1)
demo.x(0); demo.barrier()
for g in gates:
    getattr(demo, g)(0); demo.barrier()
show(demo.draw("mpl"))
```

OUTPUT:

```
=== X gate ===
Matrix (Operator):
[[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
X|0> = [0.+0.j 1.+0.j]
X|1> = [1.+0.j 0.+0.j]

=== Y gate ===
Matrix (Operator):
[[0.+0.j 0.-1.j]
 [0.+1.j 0.+0.j]]
Y|0> = [0.+0.j 0.+1.j]
Y|1> = [0.-1.j 0.+0.j]

=== Z gate ===
Matrix (Operator):
[[ 1.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j]]
Z|0> = [1.+0.j 0.+0.j]
Z|1> = [ 0.+0.j -1.+0.j]

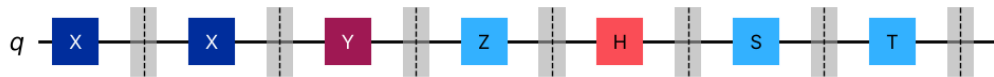
=== H gate ===
```



```
Matrix (Operator):
[[ 0.707+0.j  0.707+0.j]
 [ 0.707+0.j -0.707+0.j]]
H|0> = [0.707+0.j  0.707+0.j]
H|1> = [ 0.707+0.j -0.707+0.j]

=== S gate ===
Matrix (Operator):
[[1.+0.j 0.+0.j]
 [0.+0.j 0.+1.j]]
S|0> = [1.+0.j 0.+0.j]
S|1> = [0.+0.j 0.+1.j]

=== T gate ===
Matrix (Operator):
[[1.  +0.j  0.  +0.j  ]
 [0.  +0.j  0.707+0.707j]]
T|0> = [1.+0.j 0.+0.j]
T|1> = [0.  +0.j  0.707+0.707j]
```



Inference: Every Operator matrix matches the theory: X swaps $|0\rangle$ and $|1\rangle$, Z flips the sign of $|1\rangle$, Y is iXZ , H creates equal superpositions ($\pm$), and S, T add phases $i = e^{i\pi/2}$ and $e^{i\pi/4}$ to $|1\rangle$ only.

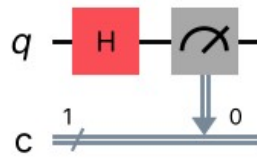
Program 2: Create a single-qubit circuit with a Hadamard gate, measure it with 1024 shots on the Aer simulator and plot the histogram. Explain why the two counts are close to, but not exactly, 512.

Code:

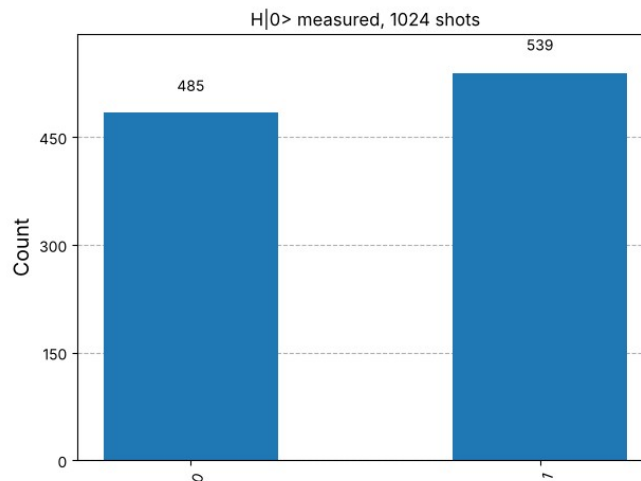
```
# Program 2: Hadamard + measurement, 1024 shots
qc = QuantumCircuit(1, 1)
qc.h(0)
print("Statevector before measurement:", np.round(Statevector(qc).data, 3))
qc.measure(0, 0)
show(qc.draw("mpl"))
counts = run(qc, 1024)
print("Counts:", counts)
for k in sorted(counts):
    print(f" P({k}) = {counts[k]/1024:.3f}")
show(plot_histogram(counts, title="H|0> measured, 1024 shots"))
```

OUTPUT:

```
Statevector before measurement: [0.707+0.j 0.707+0.j]
```



Counts: {'0': 485, '1': 539}
 $P(0) = 0.474$
 $P(1) = 0.526$



Inference: $H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$, so each outcome has probability 0.5. The counts are close to 512 but not exactly 512 because each shot is an independent random event; the deviation (statistical shot noise $\approx \sqrt{1024 \cdot 0.25} = 16$) shrinks relative to the total as the number of shots grows.

Program 3: Implement the CNOT gate and verify its truth table for all four input states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. Display its matrix with Operator and explain how it relates to the textbook matrix.

Code:

```
# Program 3: CNOT truth table. control = qubit 0, target = qubit 1
print("control target -> control target    (measured string is q1q0)")
for c in "01":
    for t in "01":
        qc = QuantumCircuit(2, 2)
        if c == "1": qc.x(0)
        if t == "1": qc.x(1)
        qc.cx(0, 1)
        qc.measure([0, 1], [0, 1])
        out = list(run(qc).keys())[0]
        print(f"    {c}      {t}      ->    {out[1]}      {out[0]}")

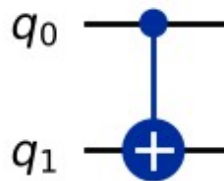
cnot = QuantumCircuit(2); cnot.cx(0, 1)
show(cnot.draw("mpl"))
print("\nOperator of cx(0,1) in Qiskit's little-endian order:\n",
      Operator(cnot).data.real)
textbook = np.array([[1,0,0,0],[0,1,0,0],[0,0,0,1],[0,0,1,0]])
print("\nAfter reversing qubit order (control written first):\n",
```



```
Operator(cnot).reverse_qargs().data.real)
print("Equal to textbook CNOT matrix:",
np.allclose(Operator(cnot).reverse_qargs().data, textbook))
```

OUTPUT:

control	target	->	control	target	(measured string is q1q0)
0	0	->	0	0	
0	1	->	0	1	
1	0	->	1	1	
1	1	->	1	0	



Operator of cx(0,1) in Qiskit's little-endian order:

```
[[1. 0. 0. 0.]
 [0. 0. 0. 1.]
 [0. 0. 1. 0.]
 [0. 1. 0. 0.]]
```

After reversing qubit order (control written first):

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 0. 1.]
 [0. 0. 1. 0.]]
```

Equal to textbook CNOT matrix: True

Inference: The target flips only when the control is 1 ($|a,b\rangle \rightarrow |a,a \oplus b\rangle$). Qiskit's matrix has rows in a different order because qubit 0 (the control) is the least-significant bit; after reversing the qubit order it is exactly the textbook CNOT matrix.

Program 4: Implement the CZ and SWAP gates and verify each on all four basis states. Show, by comparing operators, that a CNOT with a Hadamard gate on the target before and after it is equal to a CZ gate.

Code:

```
# Program 4: CZ and SWAP on all four basis states, and H-CX-H = CZ
for name in ["cz", "swap"]:
    print(f"=== {name.upper()} === input (q1q0) -> output state")
    for s in ["00", "01", "10", "11"]:
        qc = QuantumCircuit(2)
        if s[1] == "1": qc.x(0)
        if s[0] == "1": qc.x(1)
        getattr(qc, name)(0, 1)
        sv = Statevector(qc)
        i = int(np.argmax(abs(sv.data)))
        print(f" |{s}> -> {sv.data[i].real:+.0f} |{i:02b}>")
```



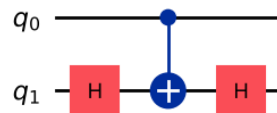
```
print()

lhs = QuantumCircuit(2); lhs.h(1); lhs.cx(0, 1); lhs.h(1)
rhs = QuantumCircuit(2); rhs.cz(0, 1)
show(lhs.draw("mpl"))
print("Operator(H.CX.H):\n", Operator(lhs).data.real.round(3))
print("Operator(CZ):\n", Operator(rhs).data.real)
print("H-CX-H equivalent to CZ:", Operator(lhs).equiv(Operator(rhs)))
```

OUTPUT:

```
=== CZ ===   input (q1q0) -> output state
|00> -> +1 |00>
|01> -> +1 |01>
|10> -> +1 |10>
|11> -> -1 |11>

=== SWAP ===   input (q1q0) -> output state
|00> -> +1 |00>
|01> -> +1 |10>
|10> -> +1 |01>
|11> -> +1 |11>
```



```
Operator(H.CX.H):
[[ 1.  0.  0.  0.]
 [ 0.  1.  0.  0.]
 [ 0.  0.  1.  0.]
 [ 0.  0.  0. -1.]]
Operator(CZ):
[[ 1.  0.  0.  0.]
 [ 0.  1.  0.  0.]
 [ 0.  0.  1.  0.]
 [ 0.  0.  0. -1.]]
H-CX-H equivalent to CZ: True
```

Inference: CZ leaves 00, 01, 10 unchanged and puts a -1 on $|11\rangle$ only (a phase, invisible to a plain measurement); SWAP exchanges 01 and 10. Since $HXH = Z$, a CNOT sandwiched by H on the target is exactly a CZ, confirmed by `Operator.equiv = True`.

Program 5: Prepare the four Bell states $|\Phi^+\rangle, |\Phi^-\rangle, |\Psi^+\rangle, |\Psi^-\rangle$. Display the statevector and the measurement histogram of each.

Code:

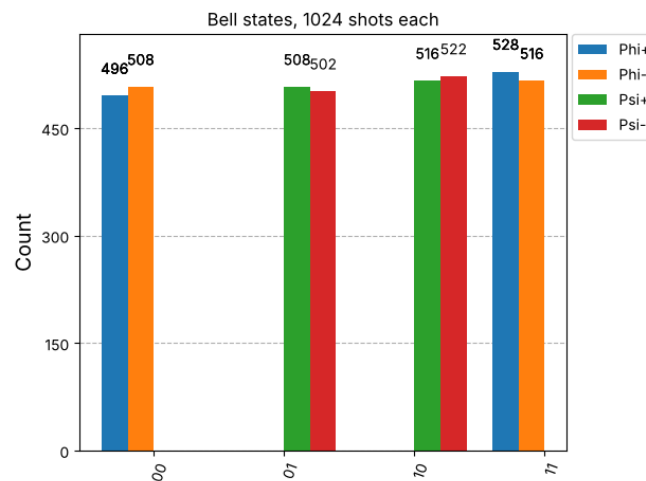
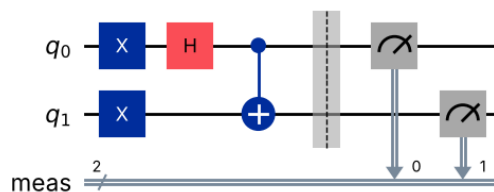
```
# Program 5: the four Bell states (input bits b1 b0 -> H on q0, CX(0,1))
names = {"00": "Phi+", "01": "Phi-", "10": "Psi+", "11": "Psi-"}
all_counts = []
for bits, nm in names.items():
    qc = QuantumCircuit(2)
    if bits[1] == "1": qc.x(0)
    if bits[0] == "1": qc.x(1)
    qc.h(0); qc.cx(0, 1)
```



```
print(f"|{nm}> (input |{bits}>):")
amps(Statevector(qc))
qc.measure_all()
all_counts.append(run(qc))
if nm == "Psi-":
    show(qc.draw("mpl"))
show(plot_histogram(all_counts, legend=list(names.values()), title="Bell states, 1024
shots each"))
```

OUTPUT:

```
|Phi+> (input |00>):
|00> : +0.707+0.000j P = 0.500
|11> : +0.707+0.000j P = 0.500
|Phi-> (input |01>):
|00> : +0.707+0.000j P = 0.500
|11> : -0.707+0.000j P = 0.500
|Psi+> (input |10>):
|01> : +0.707+0.000j P = 0.500
|10> : +0.707+0.000j P = 0.500
|Psi-> (input |11>):
|01> : -0.707+0.000j P = 0.500
|10> : +0.707+0.000j P = 0.500
```



Inference: $\Phi_{\pm}$ give only 00 and 11 (relative sign + or -), $\Psi_{\pm}$ give only 01 and 10, each with $\approx 50\%$ probability. The two qubits are perfectly correlated (Φ) or anti-correlated (Ψ): neither qubit has a state of its own, which is entanglement. The sign of Ψ^{-} appears as $(|01\rangle - |10\rangle)/\sqrt{2}$ up to a global phase.



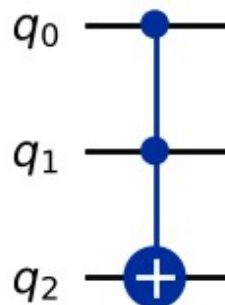
Program 6: Implement the Toffoli (CCX) gate. Verify its truth table for all eight input states and display its 8×8 matrix.

Code:

```
# Program 6: Toffoli (CCX). controls q0, q1 ; target q2
print("q2 q1 q0 (in) -> q2 q1 q0 (out)")
for i in range(8):
    qc = QuantumCircuit(3, 3)
    for b in range(3):
        if (i >> b) & 1: qc.x(b)
    qc.ccx(0, 1, 2)
    qc.measure(range(3), range(3))
    print(f" {i:03b}          -> {list(run(qc).keys())[0]}")
tof = QuantumCircuit(3); tof.ccx(0, 1, 2)
show(tof.draw("mpl"))
print("\n8x8 matrix of CCX (Qiskit order |q2 q1 q0>):\n",
      Operator(tof).data.real.astype(int))
```

OUTPUT:

```
q2 q1 q0 (in) -> q2 q1 q0 (out)
000          -> 000
001          -> 001
010          -> 010
011          -> 111
100          -> 100
101          -> 101
110          -> 110
111          -> 011
```



```
8x8 matrix of CCX (Qiskit order |q2 q1 q0>):
[[1 0 0 0 0 0 0 0]
 [0 1 0 0 0 0 0 0]
 [0 0 1 0 0 0 0 0]
 [0 0 0 0 0 0 0 1]
 [0 0 0 0 1 0 0 0]
 [0 0 0 0 0 1 0 0]
 [0 0 0 0 0 0 1 0]
 [0 0 0 1 0 0 0 0]]
```



Inference: Only inputs with both controls = 1 (011 and 111) have the target flipped (011 $\leftrightarrow$ 111); all other rows are unchanged. The 8×8 matrix is a permutation matrix that swaps the basis states $|011\rangle$ and $|111\rangle$, confirming $|a,b,c\rangle \rightarrow |a,b,c \oplus ab\rangle$.

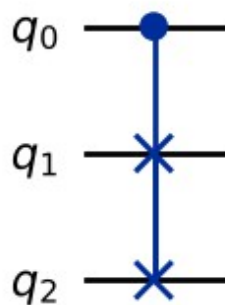
Program 7: Implement the Fredkin (CSWAP) gate. Verify its truth table for all eight input states and state which inputs are changed by the gate.

Code:

```
# Program 7: Fredkin (CSWAP). control q0 ; swaps q1 and q2
print("q2 q1 q0 (in) -> q2 q1 q0 (out)  changed?")
fred = QuantumCircuit(3); fred.cswap(0, 1, 2)
for i in range(8):
    qc = QuantumCircuit(3, 3)
    for b in range(3):
        if (i >> b) & 1: qc.x(b)
    qc.compose(fred, inplace=True)
    qc.measure(range(3), range(3))
    out = list(run(qc).keys())[0]
    print(f"    {i:03b}          ->    {out}          {'YES' if out != f'{i:03b}' else ''}")
show(fred.draw("mpl"))
```

OUTPUT:

```
q2 q1 q0 (in) -> q2 q1 q0 (out)  changed?
000          ->   000
001          ->   001
010          ->   010
011          ->   101          YES
100          ->   100
101          ->   011          YES
110          ->   110
111          ->   111
```



Inference: The two target qubits are exchanged only when the control $q_0 = 1$. Only the inputs 011 and 101 are changed (011 $\rightarrow$ 101 and 101 $\rightarrow$ 011); every other input, including 111 and 001 where the targets are equal, is unchanged. The number of 1s is always conserved.



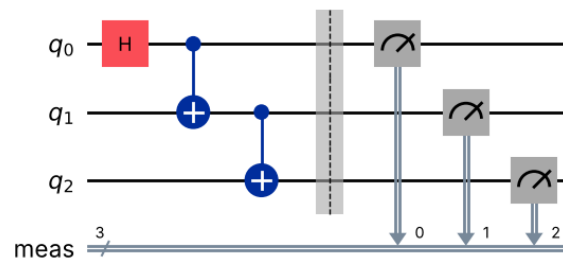
Program 8: Prepare the three-qubit GHZ state using one Hadamard gate and two CNOT gates. Display the statevector and the histogram for 1024 shots, and state which outcomes never appear.

Code:

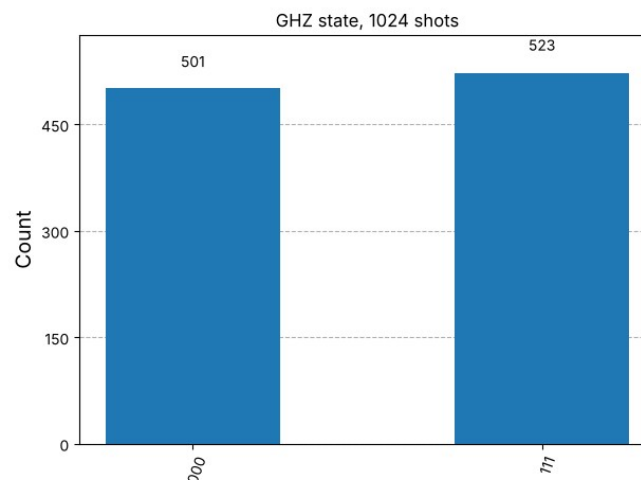
```
# Program 8: GHZ state
qc = QuantumCircuit(3)
qc.h(0); qc.cx(0, 1); qc.cx(1, 2)
print("GHZ statevector:")
amps(Statevector(qc))
qc.measure_all()
show(qc.draw("mpl"))
counts = run(qc)
print("Counts:", counts)
never = [f"{i:03b}" for i in range(8) if f"{i:03b}" not in counts]
print("Outcomes that never appear:", never)
show(plot_histogram(counts, title="GHZ state, 1024 shots"))
```

OUTPUT:

```
GHZ statevector:
|000> : +0.707+0.000j   P = 0.500
|111> : +0.707+0.000j   P = 0.500
```



```
Counts: {'111': 523, '000': 501}
Outcomes that never appear: ['001', '010', '011', '100', '101', '110']
```





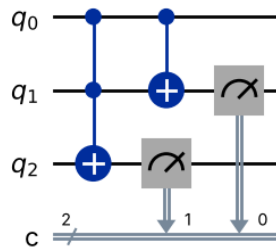
Inference: The GHZ state $(|000\rangle + |111\rangle)/\sqrt{2}$ gives only 000 and 111, each about 50%. The six outcomes 001, 010, 011, 100, 101, 110 never appear: all three qubits are always found equal, a three-qubit entangled state.

Program 9: Build a reversible half adder on three qubits using one Toffoli gate and one CNOT gate, with the third qubit starting at 0 and holding the carry. Verify the sum and carry for all four inputs.

Code:

```
# Program 9: reversible half adder. a = q0, b = q1, carry = q2 (starts at 0)
# Toffoli writes carry = a AND b, then CNOT writes sum = a XOR b into q1
ha = QuantumCircuit(3, 2)
ha.ccx(0, 1, 2)
ha.cx(0, 1)
ha.measure(1, 0) # sum -> c0
ha.measure(2, 1) # carry -> c1
show(ha.draw("mpl"))
print(" a b | sum carry")
for a in "01":
    for b in "01":
        qc = QuantumCircuit(3, 2)
        if a == "1": qc.x(0)
        if b == "1": qc.x(1)
        qc.compose(ha, inplace=True)
        out = list(run(qc).keys())[0] # c1 c0 = carry sum
        print(f" {a} {b} | {out[1]} {out[0]}")
```

OUTPUT:



a	b	sum	carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Inference: The circuit reproduces the half-adder truth table: $\text{sum} = a \oplus b$ (0,1,1,0) and $\text{carry} = a \cdot b$ (0,0,0,1). The Toffoli must act before the CNOT because it needs the original value of b; the circuit is reversible since no information is erased.

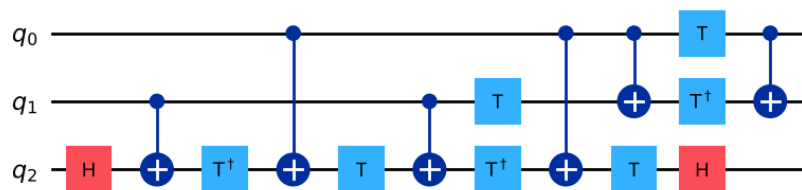


Program 10: Decompose the Toffoli gate into single-qubit and CNOT gates using `transpile` with the basis gates `h`, `t`, `tdg` and `cx`. Report the number of CNOT gates and the circuit depth, and confirm with `Operator.equiv` that the decomposed circuit is the same gate.

Code:

```
# Program 10: decompose Toffoli into {h, t, tdg, cx}
tof = QuantumCircuit(3); tof.ccx(0, 1, 2)
dec = transpile(tof, basis_gates=["h", "t", "tdg", "cx"], optimization_level=0)
show(dec.draw("mpl"))
ops = dict(dec.count_ops())
print("Gate counts:", ops)
print("Number of CNOT gates:", ops.get("cx", 0))
print("Circuit depth:", dec.depth())
print("Equivalent to CCX (Operator.equiv):", Operator(dec).equiv(Operator(tof)))
```

OUTPUT:



```
Gate counts: {'cx': 6, 't': 4, 'tdg': 3, 'h': 2}
Number of CNOT gates: 6
Circuit depth: 11
Equivalent to CCX (Operator.equiv): True
```

Inference: The Toffoli decomposes into 6 CNOT gates plus H, T and $T^\dagger$ gates (15 gates in total) and the decomposed circuit is the same gate (`Operator.equiv = True`). This is what the hardware actually runs, so a 3-qubit gate costs much more depth than a single gate.

VIVA Q&A:

1. What is a quantum gate, and why must its matrix be unitary?

Ans: A quantum gate is an operation that changes the state of one or more qubits; on n qubits it is a $2^n \times 2^n$ matrix. It must be unitary ($U^\dagger U = I$) so that the total probability stays 1 and the operation is reversible.

2. What does the Hadamard gate do to the states $|0\rangle$ and $|1\rangle$?

Ans: $H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2} = |+\rangle$ and $H|1\rangle = (|0\rangle - |1\rangle)/\sqrt{2} = |-\rangle$: it creates an equal superposition, with a minus sign for $|1\rangle$. Applying H twice gives back the original state.

3. State the truth table of the CNOT gate. Which qubit is the control and which is the target?

Ans: $00 \rightarrow 00$, $01 \rightarrow 01$, $10 \rightarrow 11$, $11 \rightarrow 10$ (control written first). The first qubit is the control and is unchanged; the second is the target and is flipped when the control is 1, i.e. $|a,b\rangle \rightarrow |a, a \oplus b\rangle$.



4. How is the CZ gate different from the CNOT gate?
Ans: CNOT flips the target bit (an X) when the control is 1, which changes the basis state. CZ only multiplies $|11\rangle$ by -1 (a phase); it is symmetric in its two qubits and its effect is invisible to a direct measurement. $CZ = (I \otimes H) \cdot CNOT \cdot (I \otimes H)$.
5. What is entanglement? How is a Bell state prepared?
Ans: Entanglement is a correlation in which the joint state cannot be written as a product of individual qubit states, so neither qubit has a state of its own. A Bell state is prepared by H on the first qubit followed by a CNOT to the second: $|00\rangle \rightarrow (|00\rangle + |11\rangle)/\sqrt{2}$.
6. What does the Toffoli gate do, and why can it reproduce any classical logic circuit?
Ans: The Toffoli (CCX) flips the target only when both controls are 1: $|a,b,c\rangle \rightarrow |a,b,c \oplus ab\rangle$. With $c = 0$ it computes AND, and with $c = 1$ it computes NAND; since NAND is universal, every classical circuit can be built reversibly from Toffoli gates.
7. What does the Fredkin gate do?
Ans: The Fredkin (CSWAP) gate swaps the two target qubits only when the control qubit is 1, otherwise it does nothing. It conserves the number of 1s and is also universal for reversible classical logic.
8. How is the GHZ state different from a Bell state?
Ans: A Bell state entangles two qubits; $GHZ = (|000\rangle + |111\rangle)/\sqrt{2}$ entangles three. In GHZ all three qubits always agree, and measuring any one qubit destroys the entanglement of the remaining two (they collapse to $|00\rangle$ or $|11\rangle$).
9. In Qiskit, which qubit is printed as the rightmost bit of a result?
Ans: Qiskit uses little-endian ordering: qubit 0 is printed as the rightmost bit of a result string, so X on qubit 0 of a two-qubit circuit prints 01.
10. Why must a three-qubit gate be decomposed before it runs on quantum hardware?
Ans: Real hardware natively supports only one-qubit and two-qubit gates (and limited connectivity), so a three-qubit gate such as Toffoli must be decomposed (transpiled) into those native gates — the standard Toffoli decomposition uses 6 CNOTs plus H, T and $T^\dagger$ gates.

RESULT:

Single-qubit (X, Y, Z, H, S, T), two-qubit (CNOT, CZ, SWAP) and three-qubit (Toffoli, Fredkin) gates were implemented in Qiskit and verified against their matrices and truth tables using Statevector, Operator and the Aer simulator (1024 shots). The four Bell states and the GHZ state were prepared and their correlated outcomes observed, a reversible half adder was built, and the Toffoli gate was decomposed into 6 CNOTs with H/T/ $T^\dagger$ gates and verified to be equivalent. Hence the experiment was successfully completed.



EVALUATION (for faculty use):

Evaluation method	Prep	Program	Output	Inference	Viva	Timely	Total
Maximum marks	1	4	2	1	1	1	10
Marks awarded							

Faculty signature with date: